Connecting your Car Dealership to a Database

Workshop 9w

Version 7.1 Y

Project Description

In this workshop you will create a Spring Boot car dealership that. You've been working on this application before, and you will convert your application into a RESTful API

Using the SpringBoot Initializer you will create a new maven project with the MySQL Driver dependency. Don't forget to add any other dependencies required for your project to your `pom.xml` file. (hint: data and web?)

NOTE: Once you have created your new project you may want to create the database in MySQL workbench, you can call it `springdealership`. You could insert some data in it as well to get you started.

Project Details

The purpose of this project is create a web based API car dealership.

Programming Process

You should spend the first few minutes setting up your project and figuring out your strategy. There are many ways to go about this, but if you are unsure, we suggest:

1. Create a new GitHub repository for this workshop named CarDealershipAPI and clone it to your `C:\pluralsight\workshops` folder
2. Go to `https://start.spring.io` and create a new Java Maven project (dealership-api)
3. Add the Spring Web, Spring Data JPA and MySQL Driver dependency
4. Create a package for the source code (com.pluralsight.dealership)
5. Generate and unzip your starter project to your repo
6. Create the models for Dealership and Vehicle, as a bonus you can add models for SalesContract and LeaseContract too
7. Build and test your program to make sure it you can connect to your API
8. Commit and push your changes.

Now you are ready to begin!

If you need any additional models to complete this workshop, add them to the project.

Phase 1: Adding configuration

Add application.properties for your database connection string

Phase 2: Adding Repositories for your models

For every model you created, add a repository. Make it an interface and make that interface extend the JPA repository interface.

Phase 3: Adding Controllers and Services

Now that you have the data layer, add the controllers and services.

VehiclesController and VehiclesService

Create one or multiple `GET` methods: that accepts path params or query string parameters for:

- `minPrice`
- `maxPrice`
- `make`
- `model`
- `minYear`
- `maxYear`
- `color`
- `minMiles`
- `maxMiles`
- `type`

Create a `POST` method: to allow a user to add a new vehicle

Create a `PUT` method: to allow the user to update vehicle information

Create a `DELETE` method: to allow the user to delete a vehicle

Add the necessary business logic to the service and make the service connect with the repository.

OPTIONAL BONUS: SalesContractsController and SalesContractsService

Create a `GET` method: to get a sales contract by Id

Create a `POST` method: to add a new Sales contract to the database

OPTIONAL BONUS: LeaseContractsController and LeaseContractsService

Create a `GET` method: to get a lease contract by Id

Create a `POST` method: to add a new Lease contract to the database

Phase 4: Implement unit tests

Last week, we also learnt how to write tests for the different layers. If time permits, add tests for the different layers of your car dealership API (repository, service and controller).

What Makes a Good Workshop Project?

- **You should:**
  - Create controllers and API endpoints for all CRUD operations
  - Implement the ability for a user to search/add/remove vehicles from the database via the endpoints

- **You should adhere to best practices such as:**
  - Create a Java Project that follows the Maven folder structure
  - Create appropriate Java packages and classes
  - Class names should be meaningful and follow proper naming conventions (PascalCase)
  - Use good variable naming conventions (camelCasing, meaningful variable names)
  - Your code should be properly formatted easy to understand
  - use Java comments effectively
  - Include a script for the Car Dealership database

- **Make sure that:**
  - Your code is free of errors and that it compiles and runs
  - Test your API endpoints using Insomnia to ensure that all methods work as expected

- **Build a PUBLIC GitHub Repo for your code**
  - Include a README.md file that describes your project and includes screen shots of
    - All cars in the database
    - Adding a car successfully via Insomnia
    - one API call that shows erroneous inputs and an error message.
  - ALSO make sure to include one interesting piece of code and a description of WHY it is interesting.